

USER_GUIDE

Setup Wizard — User Guide

A task-oriented guide to `scripts/setup-agents-wizard.sh`. Follow it top to bottom the first time; after that, jump straight to [Safe to re-run](#).

For the exhaustive reference (every function, every file, every flag) see [MANUAL.md](#). For the project overview see [README.md](#). To undo a run, see [If something goes wrong](#) and [SAFETY-AND-ROLLBACK.md](#).

Before you start

The wizard is non-interactive throughout, with exactly **one** exception: at the very end it pauses for Enter so the [ACTION REQUIRED](#) list cannot scroll past unread. That pause is skipped when stdin is not a terminal (pipes, CI) or when `WIZARD_NONINTERACTIVE=1` is set. It **can** also prompt for a sudo password when it installs `jq`, installs Ollama, or enables the `ollama` systemd service — that prompt comes from `sudo` itself. Run it from a terminal you are watching, not from a background job.

Requirement	Needed for	If missing
git	SuperSpec checkout, Claude marketplace plugin clones	Hard stop. Step 1 prints <code>git missing</code> and exits 1.
curl	Bun, ashlr and Ollama installers; embedding-backend health check	Hard stop. Step 1 prints <code>curl missing</code> and exits 1.
node	Runs the npm-installed agent CLIs	Hard stop. Step 1 prints <code>Node.js missing</code> and exits 1.
npm		

Requirement	Needed for	If missing
	Installs CodeGraph, Glyphdown and the Qwen Code CLI	Hard stop. Step 1 prints <code>npm missing</code> and exits 1. (SpecKit is a Python tool installed with <code>uv</code> , not <code>npm</code> — see Step 2.)
jq	Merging MCP servers into every agent config	Auto-installed. Linux: <code>sudo apt-get install jq</code> or <code>sudo yum install jq</code> . macOS: <code>brew install jq</code> (Homebrew is installed first if absent). If neither package manager exists the wizard exits 1.
sudo	The jq install above; the Ollama installer; <code>systemctl enable --now ollama</code>	Without it those three steps fail. The wizard degrades to warnings rather than stopping — but Lumen indexing will not work until Ollama runs.
Claude Code Lumen plugin	Provides the actual lumen binary	Step 3 prints Lumen launcher is not runnable. Install it inside Claude Code with <code>/plugin</code> , then re-run the wizard.
Internet access	Every download above	Steps degrade to warnings; re-run when connected.

Supported platforms: Linux (`apt-get` or `yum`) and macOS. The wizard branches on `$OSTYPE` and refuses to install `jq` on anything else.

Lumen is not an npm package. It ships as a Claude Code plugin binary under `~/.claude-shared/plugins/cache/`. Nothing you do with npm will install it. The wizard installs a small wrapper on your PATH that finds that binary for you.

Quick start

```
# 1. Run the wizard (from anywhere – it finds the project root
    itself)
./scripts/setup-agents-wizard.sh

# 2. Pick up the new PATH
exec bash -l          # or just open a new terminal

# 3. Index this project once, then verify the vectors are
    trustworthy
./scripts/lumen-reindex.sh "$PWD"
./scripts/lumen-index-doctor.sh "$PWD"      # exit 0 = healthy
```

Step 3 used to read `lumen index "$PWD"`, which still works. The wrapper is better: it refuses to run on an embedding backend known to write corrupt vectors, retries around transient faults, and the doctor tells you afterwards whether the result is actually usable. See [OPERATIONAL - SCRIPTS.md](#).

The wizard also **pauses for Enter** at the very end, to show you the steps it cannot perform itself. Set `WIZARD_NONINTERACTIVE=1` if you are running it from a script.

If the script is not executable yet: `chmod +x scripts/setup-agents-wizard.sh`.

The wizard determines the project root as the **parent of the scripts/ directory that contains it**, so it does not matter what your working directory is when you launch it.

What each step does, and what you will see

Output uses four markers: success, information, warning (the wizard keeps going), error. Only Step 1 can turn an into an abort.

Step 1 — Checking System Prerequisites

```
=====
Step 1: Checking System Prerequisites
=====
```

```
* All core prerequisites met.
```

Checks git, curl, node, npm, then installs jq if it is missing.

If you see	Do this
git missing (or curl/Node.js/ npm) followed by Please install missing prerequisites and re-run.	Install the named tool with your package manager and run the wizard again. The script has exited 1 and changed nothing.
jq not found – installing it now. then a sudo prompt	Enter your password. This is the only mandatory privileged step.
Could not install jq/ Unsupported OS	Install jq by hand, then re-run.

Step 2 — Installing / Updating Global CLI Tools

Installs Bun, then **only the tools that are actually missing**, by verified package name: @colbymchenry/codegraph, glyphdown and @qwen-code/qwen-code from npm, and specify via `uv tool install specify-cli`. Then ashlr, then optionally WOZCODE.

```
* Installing core utilities: CodeGraph, Glyphdown...
* codegraph already present - leaving the existing install alone.
* Installing glyphdown (provides 'glyphdown')...
* glyphdown installed from glyphdown.
* Checking AI Agent CLIs...
  Kimi not installed - it ships its own installer: https://
kimi.moonshot.cn
  ashlr-plugin installed.
  WOZCODE not installed. Set WOZCODE_INSTALL_CMD to auto-
install.
```

Two things worth knowing about this step:

- **A tool that is already on PATH is never touched.** You get `<cmd>` already present - leaving the existing install alone. and no reinstall, so the wizard cannot break a version you rely on.
- **Kimi, Opendcode and MiMo Code are detected, not installed.** The bare npm names are not these agents — opencode returns 404, kimi is a state-animation library and mimo a minimal mobile build tool.

The wizard points you at the vendor instead of installing something unrelated that merely shares the name.

If you see	Do this
<pre>Kimi / Opendcode / MiMo Code not installed - it ships its own installer: <url></pre>	Expected if you do not use that agent. To use it, install from the vendor URL and re-run the wizard so it can write that agent's MCP config.
<pre>Could not install <pkg> - '<cmd>' stays unavailable.</pre>	The npm install failed. Fix the npm problem and re-run, or install by hand. Nothing is recorded for a failed install, so rollback will never try to uninstall it.
<pre>specify missing and uv unavailable - install uv, then: uv tool install specify-cli</pre>	SpecKit is a Python tool. Install uv, then re-run — or run that command yourself.
<pre>Bun failed.</pre>	ashlr is skipped too (it needs Bun). Install Bun from https://bun.sh and re-run.
<pre>ashlr-plugin may need restart.</pre>	The installer ran but ashlr is not on PATH yet. Open a new terminal and check <code>command -v ashlr</code> .
<pre>WOZCODE not installed.</pre>	Expected unless you set <code>WOZCODE_INSTALL_CMD</code> (see Environment variables). <code>WOZCODE</code> is entirely optional.

Nothing in this step is fatal — Lumen, the part that matters most, is set up in Step 3.

Step 3 — Lumen Semantic Search Setup

The important step. It does six things in order: turns telemetry off, writes the lumen launcher, generates bash completions, wires up your shell files, installs and starts the embedding backend, then verifies the result.

```
Telemetry opt-out exported in /home/you/.bashrc
CodeGraph telemetry disabled (buffered data deleted).
Qwen usage statistics disabled.
Lumen launcher installed: /home/you/.local/bin/lumen
```

```

    Bash completions installed (lazy-loaded, no shell-startup
cost).
    Configured /home/you/.bashrc (PATH + completions).
    Configured /home/you/.bash_profile (login-shell PATH).
    Ollama present.
    ollama service active (and enabled at boot).
    Embedding model present: ordis/jina-embeddings-v2-base-code
lumen 0.0.41 responds at /home/you/.local/bin/lumen
lumen resolves on PATH in a login shell.
    Embedding backend healthy at http://localhost:11434 (round-trip
returned a vector).

```

That last line is a **real embedding round-trip**, not a ping. `/api/tags` answers 200 even when the ollama runner has wedged and returns NaN for every input, so the wizard sends an actual `health check` string to `/api/embed` and confirms a vector comes back.

If you see	What it means	Do this
Lumen launcher is not runnable - is the Claude Code Lumen plugin installed?	The wrapper exists but found no plugin binary underneath it.	Open Claude Code, run <code>/plugin</code> , install Lumen , then re-run the wizard. Or point at a binary you already have: <code>export LUMEN_BIN=/path/to/lumen-linux-amd64</code> .
Could not generate Lumen bash completions.	Same root cause — the launcher could not run.	Fix the above; completions are regenerated on the next run.
lumen not on PATH in a login shell - open a new terminal to pick it up.	<code>~/.bashrc /</code> <code>~/.bash_profile</code> were just edited; this shell predates them.	Open a new terminal, or <code>exec bash -l</code> . Harmless.
Ollama unavailable - Lumen cannot index or search without it.	The installer could not run (no sudo, no network, or unsupported platform).	Install from https://ollama.com/download , then re-run the wizard. Indexing cannot work at all without it.
	The binary is installed but the	<code>sudo systemctl enable --now ollama</code> , or run

If you see	What it means	Do this
ollama service is not active - start it before indexing.	daemon is not running.	ollama serve in a spare terminal.
Model pull failed - Lumen indexing stays broken until it succeeds.	The ~320 MB embedding model did not download.	Start Ollama, then ollama pull ordis/jina-embeddings-v2-base-code by hand.
Embedding backend unreachable at <host> - Lumen cannot index.	Nothing answered on <host>/api/tags within 5 s.	Start the daemon. If it listens elsewhere, export OLLAMA_HOST before re-running.
Embedding backend is WEDGED at <host> - it returns NaN for every input. followed by Fix: ollama stop <model>	The daemon answers, but /api/embed returns NaN instead of a vector — a known ollama runner failure under sustained load. Indexing would abort with what looks like a Lumen usage error.	Run the printed ollama stop <model>; a fresh runner clears it. Then re-run the wizard or just lumen index.
Embedding round-trip failed at <host> - Lumen will fail to index.	/api/embed answered, but with neither a vector nor NaN — usually the model is missing or still loading (the probe allows 90 s).	Check ollama list for the embedding model and pull it if absent, then re-run.
Lumen setup finished with warnings (see above).	Summary line — one of the checks above failed.	Scroll up and fix whichever one it was.
WIZARD_KEEP_TELEMETRY set - leaving telemetry settings alone.	You asked for the opt-out to be skipped.	Expected. Re-run without the variable to apply it.
Could not disable CodeGraph telemetry.	codegraph telemetry off	Run it yourself; the ~/.bashrc

If you see	What it means	Do this
	returned non-zero.	CODEGRAPH_TELEMETRY=0 export still applies to new shells.
Could not update / home/you/.qwen/ settings.json.	The jq rewrite produced an empty file, so the original was left untouched.	Check the file is valid JSON, then re-run.

What the telemetry step changes. It runs first, before the launcher, and is on by default — set WIZARD_KEEP_TELEMETRY=1 to skip it entirely:

- a **third** managed block in ~/.bashrc (# >>> telemetry opt-out (managed by Claude Code) >>>) exporting DO_NOT_TRACK=1 and CODEGRAPH_TELEMETRY=0;
- codegraph telemetry off, when codegraph is on PATH;
- .usageStatisticsEnabled = false in ~/.qwen/settings.json, when that file already exists and jq is available — backed up first, like every other file it edits.

Lumen itself ships **no** telemetry (its binary contains no analytics strings), so there is nothing to switch off for it. Full detail: [README.md → Telemetry Opt-Out](#).

Step 4 — Verifying CLI Availability

A roll call of eight commands:

lumen codegraph glyphdown specify kimi opencode mimo qwen. The Qwen Code binary is qwen; @qwen-code/qwen-code is the npm package that provides it.

```
lumen found
mimo not in PATH (may require terminal restart or manual
install)
```

... not in PATH here is informational. Some of these tools land in a PATH entry the current shell has not loaded yet — re-check after opening a new terminal before installing anything by hand.

ashlr is deliberately absent from this list. It is a Claude Code *plugin*, not a CLI: its installer clones into ~/.claude/plugins/cache/ashlr-marketplace/ashlr/ and creates no binary at all, so command -v ashlr failing is correct behaviour rather than a fault. The wizard checks the plugin **directory** instead and reports it in

the Claude Code plugins / optional tools summary section. An earlier revision probed PATH for it and printed a permanent, misleading ; tests A42 and A43 stop that coming back.

Step 5 — Configuring MCP Servers for Each Agent

Registers Lumen with each agent **at that agent’s real config location**, clones two Claude marketplace plugin repositories, and adds the Glyphdown hooks.

- Lumen MCP registered with Claude Code (user scope).
- Lumen registered in ~/.claude.json (default config).
- Glyphdown PreToolUse/PostToolUse hooks ensured for Claude Code.
- Backed up /home/you/.kimi-code/mcp.json
- MCP servers (Lumen + CodeGraph) configured for Kimi at /home/you/.kimi-code/mcp.json
- Lumen MCP configured for Opencode at /home/you/.config/opencode/opencode.json
- MiMo Code sees Lumen (inherited from ~/.claude.json).
- MCP servers (Lumen + CodeGraph) configured for Qwen Code at /home/you/.qwen/settings.json

Agent	Where it is written	Note
Claude Code	claude mcp add-json lumen ... -s user	Through the CLI, never by hand-editing the config. Skipped when claude mcp get lumen already succeeds. The CLI writes into the active CLAUDE_CONFIG_DIR , which is not necessarily ~/.claude.json.
(the default config)	~/.claude.json, jq-merged	Because of the line above. .mcpServers.lumen = {"type":"stdio","command":<wrapper>,"args":["stdio"]}. Only when the file already exists and jq is available; backed up under component claude first. Skipped with Lumen already present in ~/.claude.json if it is there.
Kimi	~/.kimi-code/ mcp.json	Skipped unless ~/.kimi-code exists.
Opencode	~/.config/ opencode/ opencode.json	Uses a .mcp object, not .mcpServers. Skipped unless the file already exists.
MiMo Code	(nothing MiMo-specific)	It inherits from ~/.claude.json — the row above is what configures it. The wizard

Agent	Where it is written	Note
		verifies with <code>timeout 25 mimi mcp list</code> , and says <code>MiMo Code inherits from ~/.claude.json</code> ; verify with: <code>mimi mcp list</code> when it cannot confirm.
Qwen Code	<code>~/.qwen/settings.json</code>	Skipped unless <code>~/.qwen</code> exists.

Correction. Earlier revisions of this guide said MiMo Code “has no documented MCP config file” and listed adding Lumen to it as *the one genuinely manual step*. **That was wrong.** MiMo reads MCP servers from `~/.claude.json` and labels them `claude:~/.claude.json`; there was never anything to configure by hand. What *was* missing is the mirror — without it, a session running with a non-default `CLAUDE_CONFIG_DIR` gave Claude Code Lumen and left every inheritor blind to it. Tests A44, A45, A46.

If you see	Do this
<code>claude CLI not found - skipping Claude Code MCP registration.</code>	Install the Claude Code CLI, then re-run.
<code>Could not register Lumen with Claude Code - add it manually:</code>	Run the <code>claude mcp add-json ...</code> line the wizard prints immediately below it.
<code>glyphdown not installed - skipping its Claude Code hook.</code>	Expected. A hook pointing at a missing binary would fire and fail on every tool call, so the wizard refuses to add one.
<code>WIZARD_SKIP_GLYPHDOWN_HOOK is set - glyphdown hook NOT registered.</code>	You asked for it. Glyphdown is still installed, it is just not wired into Claude Code. Re-run without the variable to register it — the wizard prints <code>Enable it later by re-running without that variable.</code>
<code>Kimi not installed (~/.kimi-code absent) - skipping its MCP config.</code>	Expected if you do not use Kimi. Install it, then re-run. Same for Qwen (<code>~/.qwen</code>) and Opencode (its config file).
<code>MiMo Code inherits from ~/.claude.json; verify with: mimi mcp list</code>	The wizard could not confirm MiMo sees Lumen — either <code>mimo</code> is not on <code>PATH</code> , or the probe took longer than its

If you see	Do this
<pre> Could not update ~/claude.json - left unchanged.</pre>	<pre> 25 s timeout. Run <code>mimo mcp list \</code> <code>grep lumen yourself</code>. If it is missing, check <code>jq -c '.mcpServers.lumen'</code> <code>~/claude.json</code>. The jq mirror failed. Your file is untouched. Add it by hand: .mcpServers.lumen = {"type":"stdio","command":"~/local/ bin/lumen","args":["stdio"]} (with the path written out in full).</pre>

Your existing keys and MCP servers in those files are preserved — every edit is a jq merge rather than a replacement, and every file is backed up and recorded in the rollback manifest first. If this step errors, it is almost always because jq is missing; that would have stopped Step 1 already.

Step 6 — Project-Level Setup

Creates submodules/, initializes SpecKit, checks out SuperSpec, and registers SuperSpec as a SpecKit dev extension.

```

submodules directory ready.
SpecKit already initialized.
SuperSpec already present (git submodule).
SuperSpec extension added (or already present).
```

If you see	Do this
<pre> SpecKit CLI missing – skipping project init.</pre>	<pre> uv tool install specify-cli, then re-run. SpecKit is a Python tool; there is no @specify/cli npm package.</pre>
<pre> SpecKit init may have failed.</pre>	<pre> Run specify init yourself in the project root and read its error. Expected when a previous run left a plain directory there. A registered git submodule is never deleted — the wizard detects the gitlink file and leaves it alone.</pre>
<pre> Removing stale non-git directory at submodules/ superspec</pre>	<pre> Network or auth problem. Fix it, then git submodule update --</pre>

If you see	Do this
SuperSpec submodule init failed. / SuperSpec clone failed.	init --recursive submodules/superspec.

Step 7 — Ollama Concurrency Tuning (host-specific)

This step asks `scripts/ollama-tune.sh` how ollama is actually managed on **your** machine — systemd system unit, systemd user unit, launchd, a container, a remote host, or a bare process — and what concurrency your CPU, RAM and loaded model justify. The wizard prints the delegate’s own detection and recommendation, then **stops**:

```
=====
" Step 7: Ollama Concurrency Tuning (host-specific)
=====
    Asking ./scripts/ollama-tune.sh how ollama is managed on THIS
host (report only)...
    <the delegate's detection, verbatim>
" Recommended for THIS host (computed just now by ./scripts/
ollama-tune.sh):
    <the exact commands, generated for your machine>
    NOT applied: applying RESTARTS the ollama service and kills in-
flight embeddings.
    Opt in with WIZARD_TUNE_OLLAMA=1 (you will still be asked), or
run the commands yourself.
```

Those commands also go into **ACTION REQUIRED** and **MANUAL - STEPS.md**, so you can come back to them. They are the delegate’s output for your host — the wizard never substitutes an example number for one it did not measure.

Why it will not just do it for you. Applying rewrites ollama’s config surface and restarts the service. That aborts every embedding request in flight, which means an index run that is part-way through loses that work. So three things must all be true before the wizard applies anything: you set `WIZARD_TUNE_OLLAMA=1`, no indexer is running (it checks, and treats “cannot tell” as “do not”), and you confirm at the prompt.

`WIZARD_TUNE_OLLAMA=1 ./scripts/setup-agents-wizard.sh`

If it does apply, the change is recorded in the rollback manifest as `bash <path> --revert`, together with a NOTE saying the aborted in-flight requests cannot be brought back by that revert.

If you see	What it means
Ollama tuner not present at ... - concurrency was NOT measured.	scripts/ollama-tune.sh is not in this checkout. Nothing was measured and nothing is claimed; the wizard continues.
No ollama here: not on PATH, and nothing answers at ...	There is no local CLI and no HTTP backend. Nothing to tune.
The ollama tuner reported it COULD NOT DETERMINE the answer (exit N).	The delegate could not reach a verdict. This is not “already optimal” — run it yourself and read its output.
NOT applied: an indexing job is RUNNING ...	Exactly what it says. Re-run when idle, or run the printed commands yourself when you are ready to lose the in-flight work.
Ollama concurrency already matches what this host justifies	The delegate exited 0 — nothing to recommend.

The tuner’s exit code is a three-valued verdict, not a pass/fail. 0 fine, 1 a real problem / action required, 2 could not determine. 1 is the case where there IS a recommendation — reading it as “the tuner broke” would throw that recommendation away, so the wizard maps all three explicitly and assertions K23–K25 hold it there.

Step 8 — Indexing This Project

Opt-in. By default this step does nothing at all and you see one line:

Skipping project indexing (set WIZARD_INDEX_PROJECT=1 to build indexes).

Installing the indexers does not index anything, and a first Lumen pass on a large repository is CPU-bound on the embedding backend and can run for hours — so you ask for it explicitly:

```
WIZARD_INDEX_PROJECT=1 ./scripts/setup-agents-wizard.sh
```

Then the step header prints and both indexes are built or refreshed for the project root:

```
=====
Step 8: Indexing This Project
=====
```

```
CodeGraph index exists - running incremental sync...
CodeGraph sync completed.
Lumen indexing /path/to/project (incremental; may take a long
time)...
Lumen index step finished.
```

(On a first run the two CodeGraph lines read No CodeGraph index yet - building it (this writes to .codegraph/)... and CodeGraph initial index built. instead. Failures print codegraph sync FAILED. / codegraph init FAILED.)

A here means the run finished, not that the vectors are correct. If the embedding backend was on a GPU/Vulkan path, lumen index can complete cleanly having written well-formed but *stale* vectors. Verify with ./scripts/lumen-index-doctor.sh "\$PWD" — see [Checking the index is actually correct](#).

If you see	What it means	Do this
CodeGraph index exists - running incremental sync...	.codegraph/ codegraph.db was already there, so the wizard runs codegraph sync — the incremental path.	Nothing. The wizard never runs codegraph index, which would discard the existing database first.
No CodeGraph index yet - building it (this writes to .codegraph/)... codegraph sync failed. / codegraph init failed.	First run for this project: codegraph init creates .codegraph/ in the project root. CodeGraph itself errored; only its last three output lines are shown.	Nothing. Add .codegraph/ to .gitignore if you do not want it committed. Re-run the command by hand for the full output.
codegraph not installed - skipping its index. / lumen not installed - skipping its index.	That half is simply skipped. Ollama is unreachable or the	Fix Step 2 / Step 3 first, then re-run. See Step 3's table above.

If you see	What it means	Do this
lumen index failed - check the embedding backend.	embedding model is missing.	

Neither index is recorded in the rollback manifest. Undo them with `rm -rf .codegraph` and `lumen purge "$PWD"`.

Step 9 — Provider-Side CI Verification

Read-only, and it always runs. Every other CI check in this repository looks at **files**, and a file-level check structurally cannot see a setting that lives on the provider: an organisation-default required workflow, a branch-protection required check, the GitHub Pages source setting, a provider-side scheduled export. This step asks `scripts/verify-provider-ci.sh` to go and look.

```
=====
Step 9: Provider-Side CI Verification
=====
    Querying provider settings for this checkout's remotes (read-only)...
    <the verifier's findings, verbatim>
```

The verifier's verdict has three values and the wizard keeps all three:

Verifier exit	Wizard reports
0	Provider-side CI: no provider-generated triggering found (measured just now).
1	Provider-side CI triggering CONFIRMED — plus an ACTION REQUIRED entry carrying the verifier's own findings, because turning one off means opening a provider UI
2, a timeout, or anything else	Provider-side CI status COULD NOT BE DETERMINED and Reporting UNVERIFIED. This is NOT a pass — plus an ACTION REQUIRED entry telling you how to find out (usually <code>gh auth status</code>)

A missing scripts/verify-provider-ci.sh, or a missing gh, lands in that last row too: the wizard says the status is unverified and carries on. It never reports “clean” for something it did not measure.

Final summary

The wizard closes with **six** / sections:

1. Installed Global Commands
2. Claude Code plugins / optional tools — the ashlr plugin (by directory) and WOZCODE
3. Agent MCP Configurations (Lumen)
4. Lumen Semantic Search
5. Telemetry / analytics
6. Project — SpecKit, SuperSpec, the Glyphdown hook

Read section 4 carefully: a embedding model there means search will not work no matter how green the rest is.

Every line in section 3 probes **behaviour or content**, never mere file existence — an earlier revision scored a green tick for 0-byte files, and an independent audit refuted it by replaying the predicates against empty targets. n/a means the agent is not installed; a warning further up explains any gap.

ACTION REQUIRED

After the summary comes the part the wizard cannot fake:

=====

ACTION REQUIRED – 3 step(s) only you can do

=====

A shell script cannot run Claude Code slash commands or use your sudo.

These are NOT done. Nothing above claims they are.

1. Activate the ashlr plugin inside Claude Code

```
    /plugin marketplace add ashlr/ashlr-plugin
    /plugin install ashlr@ashlr-marketplace
    /reload-plugins
    /ashlr:ashlr-status          # confirm it is live
```
2. Build the Lumen semantic index for this project

```
    ./scripts/lumen-reindex.sh "/path/to/project"
    ./scripts/lumen-index-doctor.sh "/path/to/project"  # verify afterwards
```


3. Refresh PATH in your existing terminals
 `exec bash -l` *# or just open a new terminal*

 Saved to `/path/to/project/MANUAL-STEPS.md` so you can come back to it.

Read the 3 step(s) above. Press Enter to continue...

Three things to know:

- **The list is generated from your machine's actual state**, re-evaluated on every run. Do a step, re-run the wizard, and it disappears. Nothing is ticked off by hand.
- **It is saved to `<project-root>/MANUAL-STEPS.md`**, overwritten each run. Add it to `.gitignore` if you do not want it committed; `rm` it when you are done. It is not in the rollback manifest.
- **The Enter pause is the wizard's only blocking prompt.** It is skipped when stdin is not a terminal (pipes, CI) and when `WIZARD_NONINTERACTIVE=1` is set. The list still prints and the file is still written either way.

The steps it can raise, and what each is really telling you, are documented in [`ACTION-REQUIRED.md`](#).

Verify it worked

Run these after opening a new terminal.

```
# 1. The launcher is on PATH and resolves a real binary
command -v lumen                                # => /home/you/.local/bin/
lumen version                                   # => 0.0.41 (any version
string is fine)

# 2. PATH also works in a login shell (what agents, cron and ssh
use)
bash -lc 'command -v lumen'                     # => /home/you/.local/bin/
lumen

# 3. The embedding backend answers – and actually embeds
curl -sf http://localhost:11434/api/tags >/dev/null && echo
backend-reachable
curl -s http://localhost:11434/api/embed \
-d '{"model":"ordis/jina-embeddings-v2-base-
code","input":"health check"}' \
| grep -q '"embeddings"' && echo backend-healthy # NaN here
means a wedged runner

# 4. The embedding model is present
```

```
ollama list | grep ordis/jina-embeddings-v2-base-code

# 5. Every installed agent got the MCP server, pointed at an
      absolute path with `stdio`
claude mcp get lumen
      # Claude Code
jq -c '.mcpServers.lumen' ~/.kimi-code/mcp.json ~/.qwen/
      settings.json
# => {"command":"/home/you/.local/bin/lumen","args":["stdio"]}
jq -c '.mcp.lumen' ~/.config/opencode/opencode.json
      # Opencode uses .mcp
# => {"type":"local","command":["/home/you/.local/bin/
      lumen","stdio"],"enabled":true}

# 6. Completions fire
lumen <TAB><TAB> # => completion hook index
      purge search stdio version

# 7. Telemetry is off (unless you set WIZARD_KEEP_TELEMETRY)
grep -c '>> telemetry opt-out' ~/.bashrc # -> 1
jq -r '.usageStatisticsEnabled' ~/.qwen/settings.json # -> false
jq -r '.enabled' ~/.codegraph/telemetry.json # -> false

# 8. The run was recorded and can be undone
./scripts/rollback-agents-wizard.sh --list

# 9. MiMo inherits Lumen from ~/.claude.json (there is no MiMo-
      specific config)
jq -c '.mcpServers.lumen' ~/.claude.json
mimo mcp list | grep lumen

# 10. The embedding backend is not on the corrupting Vulkan path
./scripts/ollama-vulkan-remediation.sh --check # read-only;
      expect library=cpu
```

Checking the index is actually correct

A successful lumen index proves the run finished. It does not prove the vectors are right. On a GPU/Vulkan embedding path, ollama can return **well-formed, unit-norm, correctly-sized vectors that are simply the previous vector repeated** — under HTTP 200, invisible to every per-vector check. On this project that silently corrupted 758 vectors across 55 files, and a full forensic audit passed every test it ran and declared the index trustworthy.

```
./scripts/lumen-index-doctor.sh "$PWD"
# exit 0 = healthy · 1 = corruption found · 2 = could not
      inspect
```

It is read-only and safe to run while an index is building. What it adds over the conventional checks is **aggregate distinctness**: N distinct texts must produce N distinct vectors. Nothing you can measure one vector at a time can tell you that.

If it exits 1, fix the backend first and then rebuild — `--force` is mandatory, because a corrupted file still carries a valid content hash and an incremental run skips it forever:

```
./scripts/ollama-vulkan-remediation.sh --check      #  
      library=Vulkan? then --apply  
./scripts/lumen-reindex.sh "$PWD" --force  
./scripts/lumen-index-doctor.sh "$PWD"            # expect exit  
0
```

Background: OPERATIONAL-SCRIPTS.md · INDEX-CORRUPTION-RECONCILIATION.md.

An end-to-end check, including a real index and a real semantic search against a throwaway repository:

```
./scripts/test-setup-agents-wizard.sh              # full suite,  
      including live checks  
./scripts/test-setup-agents-wizard.sh --no-live  
      # offline: static + sandboxed unit tests only
```

It writes machine-readable evidence to `.test-evidence/<UTC timestamp>/` (results.tsv, run.log, summary.json) and exits non-zero if anything failed.

If something goes wrong

Every run is recorded. Before it does anything, the wizard prints where:

```
Backup session: /home/you/.local/share/setup-agents-wizard/  
backups/20260826T210411Z  
Undo everything later with: /path/to/repo/scripts/rollback-  
agents-wizard.sh
```

Always look before you leap. `--dry-run` prints the exact plan and changes nothing:

```
./scripts/rollback-agents-wizard.sh --dry-run
```

```
=====  
Rollback plan  
=====
```

```

Session    : 20260826T210411Z
Components: all
DRY RUN - nothing will be changed.
delete    [lumen] /home/you/.local/bin/lumen
restore   [shell] /home/you/.bashrc
restore   [shell] /home/you/.bash_profile
manual    [claude] claude mcp remove lumen -s user    (use --run-
actions to execute)
restore   [kimi] /home/you/.kimi-code/mcp.json

```

When the plan looks right, apply it. --yes skips the [y/N] confirmation:

```

./scripts/rollback-agents-wizard.sh --yes
exec bash -l          # pick up the restored PATH

```

Modified files come back byte-exactly with their original permissions; files the wizard created are deleted. Undoing a run does **not** lose your current state — it is copied into <session>/pre-rollback-<UTC>/ first, so the undo is itself undoable.

Narrower fixes, when only one thing went wrong:

```

./scripts/rollback-agents-wizard.sh --list          #
    which runs can be undone
./scripts/rollback-agents-wizard.sh -c shell --yes  #
    only the ~/.bashrc + ~/.bash_profile changes
./scripts/rollback-agents-wizard.sh -c opencode --yes #
    remove Lumen from Opencode only
./scripts/rollback-agents-wizard.sh -c kimi -c qwen --yes #
    two agents, nothing else
./scripts/rollback-agents-wizard.sh --run-actions --yes #
    also undo npm installs, Speckit and the claude mcp entry

```

Two things to know:

- 1. Non-file changes are printed, not executed.** An npm install -g or the claude mcp registration shows up as manual [npm] npm uninstall -g glyphdown. Add --run-actions when you want the tool to run those for you.
- 2. One session is one run.** Rolling back the newest session returns you to the state immediately before *that* run, not to a pristine machine. To undo several runs, roll them back newest first with --session <id>.

Full reference — the manifest format, every flag, and a manual uninstall for each component if the manifest is lost: [SAFETY-AND-ROLLBACK.md](#).

For symptom-first fixes that do not need a rollback at all (lumen not found, stuck indexing, empty search results), see [TROUBLESHOOTING.md](#).

First use: Lumen

Lumen is local semantic code search. You index a project once, then ask questions in plain English instead of guessing at grep patterns.

```
# Index a project (first run builds the whole index; later runs
    are incremental)
lumen index /path/to/project

# Ask a question about it
lumen search "how are user passwords checked" -p /path/to/project

# Fewer, tighter results – locations only, no code snippets
lumen search "where is the retry policy defined" -p /path/to/
    project -n 3 --summary

# Searching from inside a subdirectory of the project
cd /path/to/project/src
lumen search "database migration runner" --cwd /path/to/project

# Throw away one project's index (or all of them, with no
    argument)
lumen purge /path/to/project
```

Things worth knowing on day one:

- **Indexing is incremental.** Re-running `lumen index <path>` only processes what changed, so it is cheap. Re-run it after a large rebase, a big merge, or edits made outside your editor. Add `-f / --force` only when you want a full rebuild from scratch.
- **The first index is the slow one.** Every chunk is embedded locally through Ollama. Large repositories take minutes; a small one takes seconds.
- **lumen search needs an index.** Run `lumen index` on the project first, or the search has nothing to match against.
- **-p / --path is the directory to search; --cwd is the project root.** Use `--cwd` when the path you are searching is a subdirectory of the indexed project.
- **stdio is for agents, not for you.** That is the MCP server subcommand the wizard put in every agent config. There is no `serve` subcommand.
- **The wizard can do the first index for you.** Re-run it with `WIZARD_INDEX_PROJECT=1` and [Step 8](#) indexes the project root with both Lumen and CodeGraph. It is opt-in precisely because that first pass can be slow.

- **A finished index is not a correct index.** Run `./scripts/lumen-index-doctor.sh <path>` after any large indexing run — see [Checking the index is actually correct](#). For long unattended runs prefer `./scripts/lumen-reindex.sh <path>`, which survives the transient backend faults that make a bare `lumen index` die halfway.
- Your agents get the same search through the Lumen MCP server automatically — you do not have to run anything for them.

Full flag reference: [MANUAL.md](#) → [Lumen CLI reference](#).

Environment variables you can set before running

These are the wizard’s own inputs — everything it reads *before or during* a run. All of them are optional; the defaults are what you get by doing nothing.

Variable	Effect	Example
WOZCODE_INSTALL_CMD	A shell command the wizard evals in Step 2 to install WOZCODE. Without it, WOZCODE is skipped with a warning. Only set this to a command you trust — it runs with your privileges.	WOZCODE_INSTALL_CMD='npm i -g wozcode' ./scripts/setup-agents-wizard.sh
LUMEN_EMBED_MODEL	Overrides the embedding model the wizard pulls and checks for. Defaults to <code>ordis/jina-embeddings-v2-base-code</code> .	LUMEN_EMBED_MODEL=nomic-embed-text ./scripts/setup-agents-wizard.sh

Variable	Effect	Example
OLLAMA_HOST	Where the wizard's health check looks for the embedding backend. Defaults to http://localhost:11434. Set it if Ollama runs on another port or host.	OLLAMA_HOST=http://127.0.0.1:11500 ./scripts/setup-agents-wizard.sh
WIZARD_INDEX_PROJECT	Opt in to Step 8. Any non-empty value makes the wizard build/refresh the CodeGraph and Lumen indexes for the project root. Unset, Step 8 prints one skip line and does nothing.	WIZARD_INDEX_PROJECT=1 ./scripts/setup-agents-wizard.sh
WIZARD_TUNE_OLLAMA	Opt in to letting Step 7 apply the ollama concurrency tuning. Unset (the default), Step 7 only reports and hands you the commands. Set, it still asks first and still refuses while an indexer is in flight — applying restarts ollama	WIZARD_TUNE_OLLAMA=1 ./scripts/setup-agents-wizard.sh

Variable	Effect	Example
	and aborts in-flight embeddings.	
OLLAMA_TUNE_SCRIPT	Path to the concurrency tuner Step 7 delegates to. Defaults to scripts/ollama-tune.sh beside the wizard. Absent = the step reports that nothing was measured.	OLLAMA_TUNE_SCRIPT=/path/to/ollama-tune.sh ./scripts/setup-agents-wizard.sh
PROVIDER_CI_SCRIPT	Path to the provider verifier Step 9 delegates to. Defaults to scripts/verify-provider-ci.sh beside the wizard. Absent = the provider status is reported UNVERIFIED, never clean.	PROVIDER_CI_SCRIPT=/path/to/verify-provider-ci.sh ./scripts/setup-agents-wizard.sh
WIZARD_KEEP_TELEMETRY	Opt out of the telemetry opt-out. Any non-empty value leaves ~/.bashrc, CodeGraph and ~/.qwen/settings.json telemetry settings exactly as they are. Unset, Step 3	WIZARD_KEEP_TELEMETRY=1 ./scripts/setup-agents-wizard.sh

Variable	Effect	Example
	disables analytics — see Step 3 .	
WIZARD_SKIP_GLYPHDOWN_HOOK	Installs Glyphdown but does not register its Claude Code hook. The hook fires on every tool call, so this lets you defer that decision without skipping the rest of Step 5. The summary then shows Glyphdown Hook skipped on request.	WIZARD_SKIP_GLYPHDOWN_HOOK=1 ./scripts/setup-agents-wizard.sh
WIZARD_STATE_DIR	Where the rollback sessions live. Defaults to \$HOME/.local/share/setup-agents-wizard. Set it identically for scripts/rollback-agents-wizard.sh , or that tool will report	WIZARD_STATE_DIR=\$PWD/.wizard-state ./scripts/setup-agents-wizard.sh
WIZARD_NONINTERACTIVE	No backup sessions found. Skip the Enter pause at the end of the ACTION REQUIRED	WIZARD_NONINTERACTIVE=1 ./scripts/setup-agents-wizard.sh

Variable	Effect	Example
	section. The section still prints and MANUAL-STEPS.md is still written — you are simply not asked to acknowledge it. The pause is skipped automatically when stdin is not a terminal, so this is only needed for automation running <i>on</i> a TTY.	
CLAUDE_CONFIG_DIR	Not set by the wizard — read by it. Defaults to ~/.claude. It decides where the wizard looks for the ashlr plugin cache and the plugins/ marketplaces/ activation marker when building the ACTION REQUIRED list, and it is where claude mcp add-json -s user actually writes. This is precisely why the wizard also mirrors Lumen	CLAUDE_CONFIG_DIR=~/.claude-work ./scripts/setup-agents-wizard.sh

Variable	Effect	Example
	into ~/.claude.json.	

The three operational scripts read a few of their own — MAX_ROUNDS and LUMEN_REINDEX_LOG for lumen-reindex.sh, LUMEN_STORE for lumen-index-doctor.sh. See OPERATIONAL-SCRIPTS.md.

Changing LUMEN_EMBED_MODEL has a cost. The model is part of an index’s identity. If the CLI and your agents disagree about the model, Lumen builds a **second index per project**. That is exactly why the wizard writes the tuning variables into ~/.bashrc **commented out** and leaves them unset. Change the model only if you mean it, and purge your old indexes (lumen purge) when you do.

Every other LUMEN_* variable is a runtime knob for the CLI, not a wizard input. See MANUAL.md → Environment variables.

Safe to re-run

Run ./scripts/setup-agents-wizard.sh as often as you like. Every step is idempotent by construction, and the test suite asserts it (groups C and D).

Step	Why re-running is safe
Prerequisites	Read-only checks. jq is installed only when absent.
Global CLIs	Every install is guarded by a “is the command already there?” check, so an existing install is left alone rather than upgraded, and nothing is installed by a guessed package name.
Lumen launcher	Backed up, then rewritten from the same template. Byte-identical unless the wizard itself changed.
Completions	Regenerated from lumen completion bash; on failure the stale file is deleted rather than left behind.

Step	Why re-running is safe
~/.bashrc / ~/.bash_profile	Each old managed block is removed before the new one is appended, so you get exactly one Lumen block and one telemetry block in ~/.bashrc, and one user-bin block in ~/.bash_profile, no matter how many times you run. Everything outside the markers is untouched, and the file's permissions and inode are preserved.
Telemetry opt-out	Idempotent by the same strip-then-append mechanism (test H2). codegraph telemetry off is safe to repeat, and ~/.qwen/settings.json is only rewritten when .usageStatisticsEnabled is not already false — otherwise you get Qwen usage statistics already disabled.
Ollama	The install is skipped if ollama is on PATH; the service is enabled only if not already active; the model is pulled only if ollama list does not show it.
Agent MCP configs	Every edit is a jq merge — re-running overwrites only the keys the wizard owns and leaves the rest of your config alone. After ten runs Kimi and Qwen still have exactly two MCP servers. Claude Code is skipped outright when claude mcp get lumen already succeeds.
Glyphdown hook	Added only when glyphdown is installed and WIZARD_SKIP_GLYPHDOWN_HOOK is unset, and guarded by an index("glyphdown") check, so re-runs neither duplicate it nor

Step	Why re-running is safe
	disturb hooks you added yourself.
Ollama tuning (Step 7)	Report-only by default, so re-running changes nothing. With WIZARD_TUNE_OLLAMA=1 the delegate is asked again for this host's current facts; if the setting is already what this host justifies there is nothing to apply.
Provider-side CI check (Step 9)	Read-only. It queries, it never writes.
Project indexing (Step 8)	Skipped entirely unless WIZARD_INDEX_PROJECT is set. When it is, both passes are incremental — codegraph sync on an existing database, lumen index re-embedding only what changed.
Claude marketplace plugins	Cloned only if the target directory does not exist.
SpecKit	Skipped entirely if .specify/ or specs/ already exists.
SuperSpec	A checked-out submodule is detected by its gitlink file and left alone. Only a stale, non-git directory is removed.

Three caveats:

- Backups accumulate.** Every run copies each config it touches to `<file>.bak.<YYYYMMDDHHMMSS>`. After many runs you will have a pile of them next to `~/.kimi-code/mcp.json` and friends. `~/.bashrc` collects **two** per run, because both `configure_telemetry_optout` and `configure_lumen_shell` back it up (the manifest still holds a single row for it — the first snapshot wins). Prune them when you are confident: `rm -f ~/.bashrc.bak.* ~/.claude/*.bak.* ~/.kimi-code/*.bak.* ~/.qwen/*.bak.*`
- Rollback sessions accumulate too.** Each run opens a new one under `~/.local/share/setup-agents-wizard/backups/`. They are small — only the files that changed — but they are never pruned automatically. `--list` shows them all.

3. **Re-running does not undo your edits.** If you hand-edited the managed block in `~/.bashrc`, the next run replaces it. Put your own customizations *outside* the markers.
4. **MANUAL-STEPS.md is overwritten every run**, and is not in the rollback manifest. It is a snapshot of what is *still* pending, not a checklist you tick off — the list shrinks by itself as you complete the steps and the wizard's detection stops firing. Do not edit it and expect the edit to survive; `rm` it when you are done.

To undo the shell changes, roll back the `shell` component — it restores both files byte-exactly, markers and all:

```
./scripts/rollback-agents-wizard.sh --component shell --dry-run  
./scripts/rollback-agents-wizard.sh --component shell --yes
```

By hand instead, delete everything between the marker lines — see [MANUAL.md → Uninstalling cleanly](#).